HW11

Nathan Krieger

2026-04-24

Problem 1: Concept Review

(a)

Suppose we collect data for a group of students in an economics class with variables X1 = hours studied, X2 = undergrad GPA, and Y = receive an A or not. We fit a logistic regression model and produce estimated regression coefficients, $\hat{\beta}_0$ = -6, $\hat{\beta}_1$ = 0.05, and $\hat{\beta}_2$ = 1.

i.

Estimate the probability that a student who studies for 40 hours and has an undergrad GPA of 3.5 gets an A in the class.

```
b_0 <- -6
b_1 <- 0.05
b_2 <- 1

Y <- b_0 + b_1 * 40 + b_2 * 3.5

p <- exp(Y)/(1 + exp(Y))
p
```

```
## [1] 0.3775407
```

37.7%

ii.

How many hours would the student in part (i) need to study to have a 50% chance of getting an A in the class?

$0 = -6 + 0.05x + 1(3.5)$

$2.5 = 0.05x$

$x = 50$

They would need to study 50 hours.

(b)

If the true decision boundary between two groups is linear and the constant variance assump- tion holds, do you expect LDA or QDA to perform better on the testing set? Explain using concepts from bias/variance tradeoff.

LDA would perform better since QDA would over fit the data. Since the true model is linear LDA would have near 0 bias. QDA would have a low bias too but it is more flexible causing it to have a higher variance.

(c)

Same setup as (b), do you expect LDA or QDA to perform better on the training set? Justify your answer.

QDA would perform better on the training set because it has more flexibility and can over fit to the training data.

(d)

For each of the following classification problems, state whether you would advise a client to use LDA, logistic regression, or KNN and explain why:

i.

We want to predict gender based on height and weight. The training set consists of heights and weights for 82 men and 63 women.

LDA because height and weight follow a normal distribution. LDA tends to be more stable than logistic regression.

ii.

We want to predict gender based on annual income and weekly working hours. The training set consists of 770 men and 820 women.

Logistic regression because income is not expected to be normally distributed. LDA assumes a normal distribution.

iii.

We want to predict gender based on a set of predictors where the decision boundary is complicated and highly non-linear. The training set consists of 960 men and 1040 women.

KNN because LDA and logistic regression are both linear classifiers.

Problem 2: k-NN

Assume our outcome Y can take on Y = 0, Y = 1 or Y = 2 (3 categories). Suppose we have a training data set with 5 observations. We want to classify a test observation using KNN. Below are all the distances between each of the 5 observations in training set and the test observation.

Training Observation (i): 1 2 3 4 5 Yi label: 0 1 2 0 2 distance: 0.45 0.23 0.61 0.31 0.57

(a)

Based on the above, how would we classify our test observation using K = 1?

Y = 1

(b)

How would we classify our test observation using K = 3?

Y = 0

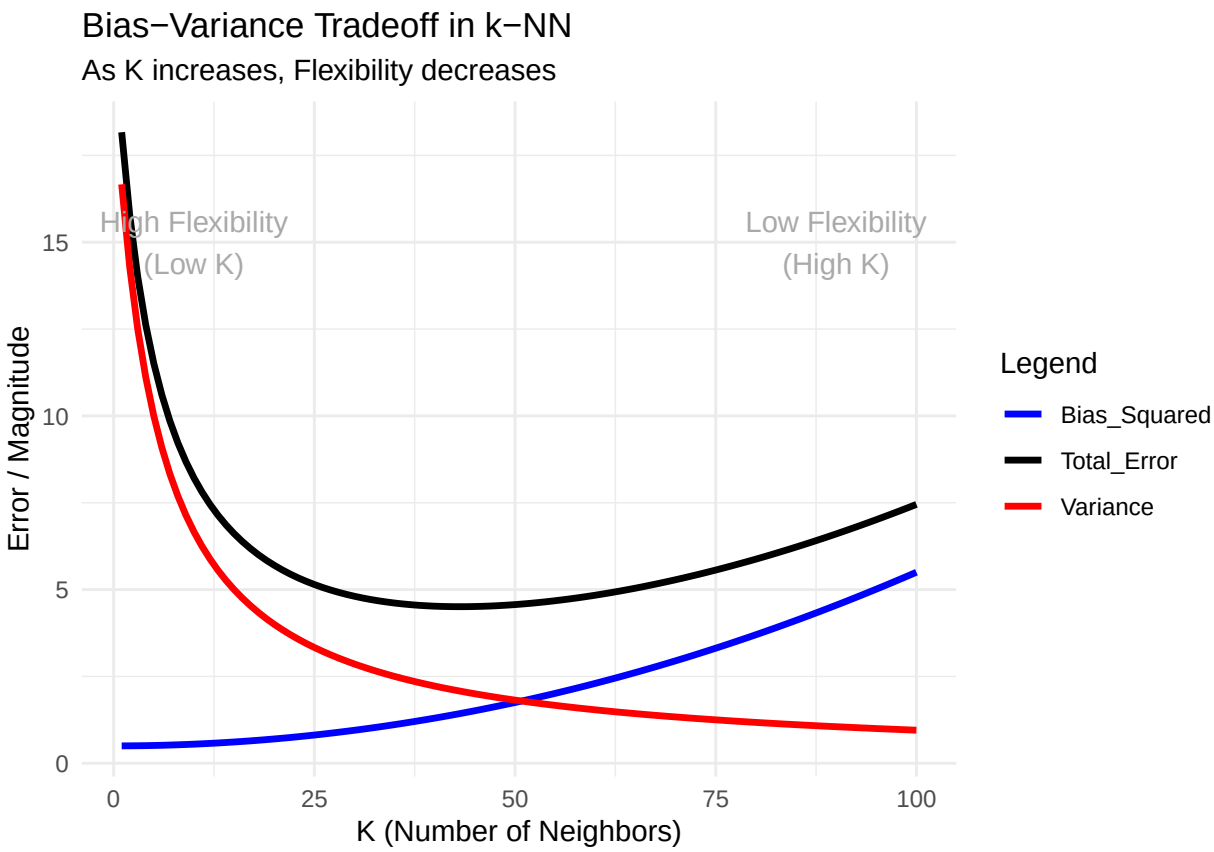
(c)

KNN is highly dependent on the choice of K. Does the classifier become more flexible or less flexible as K increases? Draw a plot with K on the x-axis and bias (squared) /variance on the y-axis. How does the bias/variance behave as K increases?

Less flexible as K increases. So bias increases, variance decreases.

```r
library(ggplot2)
library(tidyr)

# 1. Create simulated data representing the theoretical curves
# As K increases, flexibility decreases.
k_values <- 1:100

# Variance decreases as K increases (sturdier model)
variance <- 100 / (k_values + 5)

# Bias^2 increases as K increases (simpler model)
bias_sq <- 0.0005 * (k_values^2) + 0.5

# Total error is the sum of the two (plus some irreducible error/noise)
total_error <- variance + bias_sq + 1

# 2. Combine into a data frame
df <- data.frame(
  K = k_values,
  Variance = variance,
  Bias_Squared = bias_sq,
  Total_Error = total_error
)

# 3. Pivot the data for easier plotting with ggplot
df_long <- pivot_longer(df, cols = -K, names_to = "Metric", values_to = "Value")

# 4. Generate the plot
ggplot(df_long, aes(x = K, y = Value, color = Metric)) +
  geom_line(size = 1.2) +
  scale_color_manual(values = c("Bias_Squared" = "blue",
                                "Variance" = "red",
                                "Total_Error" = "black")) +
  labs(title = "Bias-Variance Tradeoff in k-NN",
       subtitle = "As K increases, Flexibility decreases",
       x = "K (Number of Neighbors)",
       y = "Error / Magnitude",
       color = "Legend") +
  theme_minimal() +
  annotate("text", x = 10, y = 15, label = "High Flexibility\n(Low K)", color = "darkgray") +
  annotate("text", x = 90, y = 15, label = "Low Flexibility\n(High K)", color = "darkgray")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Problem 3: Email Spam Part 2

Use the Spam data set, from HW 9, for this problem. Repeat your code from Problem 3 part (b).

(a)

What type of mistake do we think is more critical here: reporting a meaningful email as spam (false positive) or a spam email as meaningful (false negative)?

A false positive is more critical. Reporting a non-spam email as spam is bad so we want to keep false positive rates low.

(b)

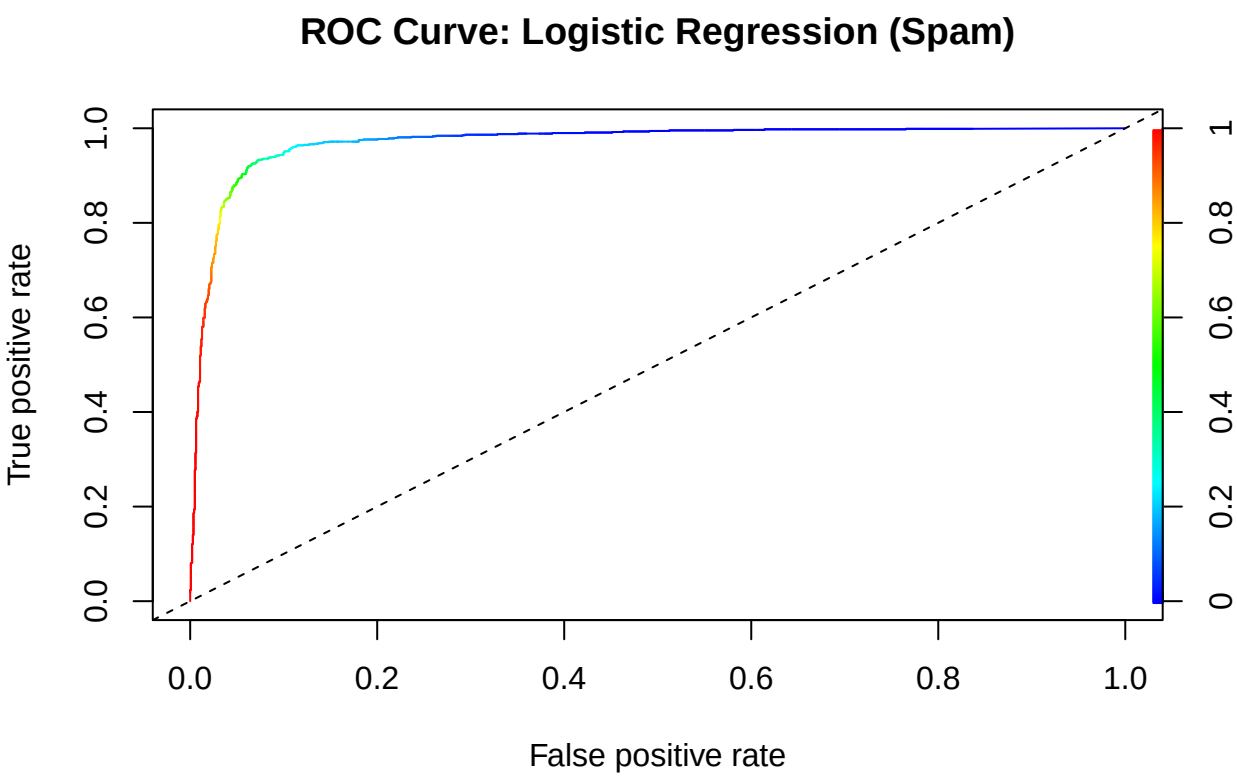
Fit a logistic regression model here and apply it to the test set. Based on your answer to part (a), plot the ROC curve of true positive rate vs. false positive rate or true negative rate vs. false negative rate.

```
#install.packages("ROCR")
library(ROCR)

spam = read.csv("spambase.data", header = FALSE)

set.seed(1)

n <- nrow(spam)

train_index <- sample(1:n, n / 2, replace = FALSE)
```

```r
train <- spam[train_index, ]
test  <- spam[-train_index, ]

glm_fit <- glm(V58 ~ ., data = train, family = "binomial")
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
glm_prob <- predict(glm_fit, newdata = test, type = "response")

ROCRpred <- prediction(glm_prob, test$V58)
perf <- performance(ROCRpred, "tpr", "fpr")

plot(perf, colorize = TRUE, main = "ROC Curve: Logistic Regression (Spam)")
abline(a = 0, b = 1, lty = 2) # Diagonal line
```



(c)

Output the confusion matrix. What is the false positive and false negative rate when we set the threshold to be 0.5?

```r
glm_pred <- rep(0, nrow(test))
glm_pred[glm_prob > 0.5] <- 1

# Confusion Matrix
conf_matrix <- table(Predicted = glm_pred, Actual = test$V58)
print(conf_matrix)
```

```
##           Actual
## Predicted    0    1
##         0 1337   91
```

```
##         1   78  795
```

```r
# Rates
fpr <- conf_matrix[2,1] / sum(conf_matrix[,1]) # FP / Total Negatives
fnr <- conf_matrix[1,2] / sum(conf_matrix[,2]) # FN / Total Positives

cat("Threshold 0.5 -> FPR:", fpr, "| FNR:", fnr)
```

```
## Threshold 0.5 -> FPR: 0.05512367 | FNR: 0.1027088
```

(d)

Adjust the threshold such that your chosen error (false positive or false negative) is no more than 0.03. You should choose the threshold carefully so that the true positive and true negative rate are also maximized. Report that threshold here.

```r
thresholds_df <- data.frame(
  threshold = perf@alpha.values[[1]],
  fpr = perf@x.values[[1]],
  tpr = perf@y.values[[1]]
)

safe_thresholds <- subset(thresholds_df, fpr <= 0.03)
best_row <- safe_thresholds[nrow(safe_thresholds), ]

print(best_row)
```

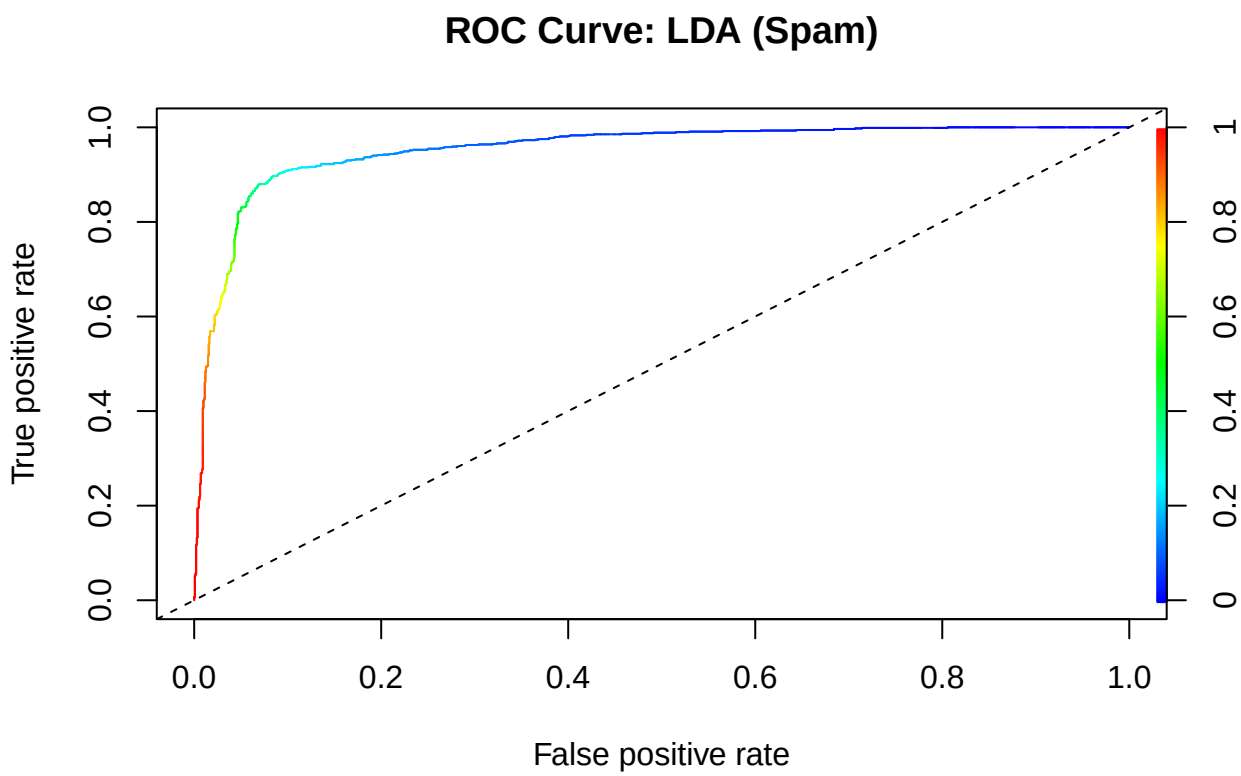
```
##     threshold        fpr      tpr
## 693 0.7858869 0.02968198 0.785553
```

0.7858869

(e)

Implement LDA and repeat parts (b) – (d).

```r
library(MASS)

# (b) Fit LDA and plot ROC
lda_fit <- lda(V58 ~ ., data = train)
lda_probs <- predict(lda_fit, newdata = test)$posterior[, 2] # Prob for Spam

ROCRpred_lda <- prediction(lda_probs, test$V58)
perf_lda <- performance(ROCRpred_lda, "tpr", "fpr")

plot(perf_lda, colorize = TRUE, main = "ROC Curve: LDA (Spam)")
abline(a = 0, b = 1, lty = 2)
```




```r
# (c) Confusion Matrix at 0.5 threshold
lda_pred_50 <- rep(0, nrow(test))
lda_pred_50[lda_probs > 0.5] <- 1

conf_matrix_lda <- table(Predicted = lda_pred_50, Actual = test$V58)
print(conf_matrix_lda)
```

```
##          Actual
## Predicted    0    1
##         0 1349  180
##         1   66  706
```

```r
fpr_lda <- conf_matrix_lda[2,1] / sum(conf_matrix_lda[,1])
fnr_lda <- conf_matrix_lda[1,2] / sum(conf_matrix_lda[,2])
cat("LDA 0.5 Threshold -> FPR:", fpr_lda, "| FNR:", fnr_lda, "\n")
```

```
## LDA 0.5 Threshold -> FPR: 0.04664311 | FNR: 0.2031603
```

```r
# (d) Threshold for FPR <= 0.03
lda_thresh_df <- data.frame(
  threshold = perf_lda@alpha.values[[1]],
  fpr = perf_lda@x.values[[1]],
  tpr = perf_lda@y.values[[1]]
)

safe_lda <- subset(lda_thresh_df, fpr <= 0.03)
# We pick the one with the highest TPR (the last row of the subset)
best_lda_row <- safe_lda[nrow(safe_lda), ]
print(best_lda_row)
```

```
##     threshold        fpr       tpr
```

```
## 583 0.7299196 0.02968198 0.6455982
```

False positive rate: 0.02968

(f)

Train K-NN on the training set. You should obtain K using cross-validation: write your own for-loops to do so. Remember to standardize your predictors for KNN. Report the confusion matrix and overall test error rates for the classifier.

```r
library(class)

# Standardize predictors (excluding the response column V58)
train_x <- scale(train[, -58])
test_x <- scale(test[, -58], center = attr(train_x, "scaled:center"),
                scale = attr(train_x, "scaled:scale"))

train_y <- train$V58
test_y <- test$V58

# 10-fold CV for K
set.seed(1)
K_vals <- c(1, 3, 5, 7, 9, 11)
fold_size <- 10
folds <- cut(seq(1, nrow(train_x)), breaks = fold_size, labels = FALSE)
cv_errors <- matrix(NA, nrow = length(K_vals), ncol = fold_size)

for(i in 1:length(K_vals)) {
  for(j in 1:fold_size) {
    val_idx <- which(folds == j)
    pred <- knn(train = train_x[-val_idx, ], test = train_x[val_idx, ],
                cl = train_y[-val_idx], k = K_vals[i])
    cv_errors[i, j] <- mean(pred != train_y[val_idx])
  }
}

mean_cv_errors <- rowMeans(cv_errors)
best_K <- K_vals[which.min(mean_cv_errors)]
cat("Optimal K:", best_K, "\n")
```

```
## Optimal K: 1
```

```r
# Final evaluation on test set
knn_pred <- knn(train = train_x, test = test_x, cl = train_y, k = best_K)
conf_knn <- table(Predicted = knn_pred, Actual = test_y)
print(conf_knn)
```

```
##          Actual
## Predicted    0    1
##         0 1277  125
##         1  138  761
```

```r
cat("KNN Test Error Rate:", mean(knn_pred != test_y), "\n")
```

```
## KNN Test Error Rate: 0.1142981
```

(g)

Which classifier would you recommend for this data? Justify your answer.

I would recommend logistic regression because marking a meaningful email as spam is more consequential. Logistic regression minimized the false positive rate to 3%. LDA had a similar FPR but the FNR was much higher. KNN had a worse results than both of the other methods.